

SOFTWARE OVERVIEW & DOCUMENTATION

EventX

Complete Software Overview & Documentation — v1.0

Event/Festival Management Platform

Created by Team SynaptiX

Document Version: 1.0

Source: Artfest-Web-app-main

Live: isra-rendezvous.vercel.app

Table of Contents

01	Product Overview	10	Poster Generation System
02	Technology Stack	11	Print Sheet System
03	Routes & Pages	12	Data Import System
04	Authentication & Security	13	Gallery & Media System
05	Admin Panel — Features & Workflows	14	Database Schema & Data Model
06	Student Panel — Features & Workflows	15	Supabase RPCs & Functions
07	Judge Panel — Features & Workflows	16	Design System
08	Public Pages — Features & Workflows	17	Project Structure & Deployment
09	Green Room & Lot Draw System		

Product Overview

What is EventX?

EventX is a full-stack web application designed to manage every aspect of large-scale academic and cultural festivals. It replaces spreadsheets, paper forms, and manual coordination with a unified, role-based digital platform — from participant registration to real-time result publishing.

The platform is purpose-built for the ISRA Vatanappally Life Festival ("Rendezvous'26"), supporting 500+ participants across 20+ event categories with multiple competition teams.

User Roles

Role	Login	Session	Key Capabilities
Admin	/admin/login	Supabase Auth (email/password)	Full CRUD on all entities, result publishing, poster generation, gallery management, print sheets, lot draw
Student	/student/login	Custom: chestNo + password, localStorage + DB	View own profile, programmes, results; download posters; edit details (admin-gated)
Judge	/judges/login	Supabase Auth (separate client instance)	Submit results per programme, re-edit locked results with CAPTCHA verification
Public	None	None	View home page, gallery, results, result details, download posters
Green Room	/lots (PIN gate)	sessionStorage	Lot draw (numbers/letters), code letter assignment to participants

Key Statistics (from Source Code)

30 Routes
10 public, 13 admin, 2 judge, 1 student, 4 redirects

14 Database Tables
programmes, teams, students, results, categories, spotlight, poster_templates, gallery_footers, performance_code_assignments, student_credentials, judge_captcha_challenges, result_edit_log, admin (legacy)

4 RPCs
judge_create_captcha, judge_reverify_edit, admin_set_result_no, admin_delete_results_for_programme

26 Components
Reusable UI components including poster engine, import modals, route guards, 3D scenes

15 Admin Pages
Dashboard, programmes, teams, students, categories, results, gallery, footers, print, posters (2), lot draw

3 Supabase Clients
Main (admin), judgeClient (judge auth), verifyJudgeClient (re-verification)

Architecture Summary

EventX is a React SPA (Single Page Application) using Vite as the build tool, Supabase as the backend (PostgreSQL + Auth + Storage), and Tailwind CSS for styling. The app is deployed on Vercel with SPA catch-all routing.

All server-side logic is handled by PostgreSQL functions (RPCs) — there are no Edge Functions or separate API servers. Real-time subscriptions are not used; the app operates on a pull/refresh model.

Technology Stack

Layer	Technology	Version	Notes
UI Framework	React	19.2.7	Functional components, hooks
Build Tool	Vite	8.1.0	ESM, @vitejs/plugin-react 6.0.2
Routing	react-router-dom	7.18.0	BrowserRouter, nested routes
Backend	Supabase	2.108.2	PostgreSQL, Auth, Storage, RPCs
CSS	Tailwind CSS	3.4.19	Utility-first, CSS variable tokens
3D Graphics	Three.js + React Three Fiber	0.185.1 / 9.7.0	Hero scene with GLB models
Icons	lucide-react	1.21.0	Consistent icon system
CSV Parsing	PapaParse	5.6.0	Bulk participant/programme import
Excel Parsing	SheetJS (xlsx)	0.18.5	Bulk import from .xlsx/.xls
PDF Extraction	pdfjs-dist	6.2.108	Text extraction from PDF files
OCR	Tesseract.js	7.0.0	Image-to-text for import from images
Screenshot	html2canvas	1.4.1	Poster/sheet PNG export
File Download	file-saver	2.0.5	Client-side file download
Progress UI	react-circular-progressbar	2.2.0	Student dashboard progress ring
Linters	oxlint	1.69.0	Oxidation Compiler linter
Deployment	Vercel	—	SPA with catch-all rewrite

Architecture Decisions

Decision	Approach	Rationale
State Management	Component-local state + localStorage	No global state library (Redux/Zustand). Data fetched per-component. Session state in localStorage/DB.
Backend	Supabase (Postgres + RPCs)	No separate API server. All write logic in SECURITY DEFINER PostgreSQL functions. RLS for access control.
Rendering	Client-side SPA	All pages render client-side. Public pages are index.html + JS. No SSR/SSG.
Auth Isolation	3 Supabase client instances	Separate storageKeys prevent GoTrueClient collisions between admin, judge, and verify sessions.
Student Auth	Custom session (not Supabase Auth)	Students authenticate as anon role. Session tracked in both localStorage and DB for single-device enforcement.
Image Handling	Supabase Storage (public)	All images stored in single "photos" bucket with folder-based organization. Public read access.
Real-time	Pull/refresh (no subscriptions)	Simplicity. Admin refreshes to see updates. No WebSocket overhead.

NPM Scripts

Script	Command	Purpose
dev	vite	Development server with HMR
build	vite build	Production build to dist/
lint	oxlint	Run linter
preview	vite preview	Preview production build locally

Environment Variables

Variable	Purpose
VITE_SUPABASE_URL	Supabase project URL
VITE_SUPABASE_ANON_KEY	Supabase anonymous API key

Routes & Pages

Public Routes

Path	Component	Description
/	Home	Landing page with 3D hero, team standings, stats, gallery preview, about section
/gallery	Gallery	Photo gallery with album grouping, lightbox, download with footer overlay
/results	Results	Published results grid with category filter, unpublished toggle
/results/:id	ResultDetail	Single programme result with placements and poster download
/programmes	Redirect → /results	Legacy redirect
/programmes/:id	Redirect → /results/:id	Legacy redirect

Student Routes

Path	Component	Guard	Description
/student/login	StudentLogin	None	Chest number + password authentication
/student/dashboard	StudentDashboard	Session check (localStorage + DB)	Profile, programme progress, results, poster download

Admin Routes

Path	Component	Description
/admin/login	AdminLogin	Email/password login (redirects if session active)
/admin	AdminDashboard	Statistics, team points leaderboard, individual points by category
/admin/programmes	AdminProgrammes	CRUD programmes, toggle publish, clear results, bulk import
/admin/categories	AdminCategories	CRUD categories (name + sort order)
/admin/teams	AdminTeams	View/edit team names and colors, team breakdown
/admin/students	AdminStudents	CRUD participants, photo upload, bulk import
/admin/results	AdminResults	Read-only results preview for all programmes
/admin/result-poster	AdminResultPoster	Cumulative team standings poster generator

<code>/admin/print</code>	AdminPrint	Print sign/valuation/result sheets
<code>/admin/spotlight</code>	AdminSpotlight	Gallery image upload and management
<code>/admin/frames/templates</code>	AdminPosterTemplates	Poster template library (create, duplicate, delete)
<code>/admin/frames/templates/:id/edit</code>	AdminPosterEditor	Visual drag-and-drop poster template editor
<code>/admin/frames/footer</code>	AdminGalleryFooters	Gallery footer overlay management

All `/admin/*` routes (except `/admin/login`) are wrapped in `ProtectedRoute` which checks `supabase.auth.getSession()`.

Green Room Routes

Path	Component	Guard	Description
<code>/lots</code>	LotsAccess	6-digit PIN (201219)	Access gate for lot draw system
<code>/lots/draw</code>	LotsDraw	<code>sessionStorage('lots_granted')</code>	Lot draw container (topic/code/assign modes)

Judge Routes

Path	Component	Guard	Description
<code>/judges/login</code>	JudgesLogin	None	Judge email/password login (validates role)
<code>/judges/results</code>	JudgesResults	JudgesRoute (role check)	Result entry panel with CAPTCHA re-verification

Route Guards

Guard	File	Protects	Mechanism
ProtectedRoute	<code>src/components/ProtectedRoute.jsx</code>	<code>/admin/*</code>	<code>supabase.auth.getSession()</code> → redirect to <code>/admin/login</code>
JudgesRoute	<code>src/components/JudgesRoute.jsx</code>	<code>/judges/results</code>	<code>judgeClient.auth.getSession()</code> + <code>app_metadata.role === 'judge'</code>
AdminSessionRedirect	<code>src/App.jsx</code> (inline)	<code>/admin/login</code>	If authenticated, redirect to <code>/admin</code>
Student session guard	<code>src/pages/StudentDashboard.jsx</code>	<code>/student/dashboard</code>	<code>getStudentSessionState()</code> — checks <code>localStorage</code> + DB

Authentication & Security

Supabase Client Configuration

Three separate Supabase client instances are created in `src/supabase/client.js`, all sharing the same URL and anon key but with different `storageKey` values to prevent session collisions:

Client	Variable	storageKey	Purpose
Main	<code>supabase</code>	Default (sb-*-auth-token)	Admin auth, all data queries
Judge	<code>judgeClient</code>	artfest-judge-auth	Judge authentication (isolated session)
Verify	<code>verifyJudgeClient</code>	artfest-verify-auth	Re-verification flow (persistSession: false)

Role System

Roles are stored in `auth.users.raw_app_meta_data` and carried in the JWT as `app_metadata.role`. Two SQL helper functions are defined:

```
CREATE OR REPLACE FUNCTION public.is_admin() RETURNS boolean AS $$
  SELECT COALESCE(auth.jwt() → 'app_metadata' →> 'role', '') = 'admin';
$$;

CREATE OR REPLACE FUNCTION public.is_judge() RETURNS boolean AS $$
  SELECT COALESCE(auth.jwt() → 'app_metadata' →> 'role', '') = 'judge';
$$;
```

Admin Authentication Flow

1. User navigates to `/admin/login`
2. `AdminSessionRedirect` checks if session already exists → redirects to `/admin` if so
3. Admin enters email + password
4. `supabase.auth.signInWithPassword({ email, password })` is called
5. On success, JWT is stored with `app_metadata.role = "admin"`
6. Navigate to `/admin` with `replace: true` (no back-stack entry for login)
7. `ProtectedRoute` on all `/admin/*` routes verifies session exists

Student Authentication Flow

1. Student navigates to `/student/login`
2. Enters chest number and password
3. `getStudentByCredentials(chestNo, password)` queries the `students` table
4. Checks `student_credentials` table for password match
5. Falls back to common password `israfest2026` if no per-student credential exists
6. Checks `sessionActive` in DB — rejects with `'already_logged_in_elsewhere'` if true
7. Creates session: stores token in `localStorage` (`student_session_<id>`) and DB (`sessionActive=true, sessionToken, sessionExpiresAt`)
8. Session expires after 8 hours (`SESSION_EXPIRY_MS = 8 * 60 * 60 * 1000`)
9. Student ID stored in `localStorage.setItem('student_id', id)`

Note: Students authenticate as the `anon` role in Supabase Auth. Their writes to the students table are gated by RLS policies requiring `auth.role() = 'authenticated'` , so students cannot write directly via PostgREST.

Judge Authentication Flow

1. Judge navigates to `/judges/login`
2. Any existing judge session is cleared: `judgeClient.auth.signOut()`
3. Enters email + password
4. `judgeClient.auth.signInWithPassword({ email, password })` is called
5. After sign-in, checks `data.user.app_metadata.role`
6. If `role !== 'judge'`, signs out and shows error: "This account is not registered as a judge."
7. On success, navigates to `/judges/results`
8. JudgesRoute guard verifies session + role on every access

Result Re-Verification Flow (Judge Editing Locked Results)

1. Judge clicks "Re-verify & Edit" on a locked result
2. Prompt: "Are you really a Judge?"
3. CAPTCHA challenge generated via `judge_create_captcha` RPC (6-char code, 5-minute expiry)
4. Judge re-enters email + password
5. Enters the CAPTCHA code
6. Client calls `judge_reverify_edit` RPC with all data
7. Server-side verification: checks JWT role, validates password via `crypt()` , validates CAPTCHA (valid, unexpired, unused)
8. On success: upserts result, sets `Locked=true` , writes audit log to `result_edit_log`

Row-Level Security (RLS) Policies

Table	SELECT	INSERT	UPDATE	DELETE
students	public	authenticated	authenticated	authenticated
programmes	public	authenticated	authenticated	authenticated
results	public	admin (placeholder) / judge (locked)	!locked	judge & !locked
teams	public	authenticated	authenticated	authenticated
categories	public	is_admin()	is_admin()	is_admin()
spotlight	public	authenticated	authenticated	authenticated
poster_templates	anon + auth	authenticated	authenticated	authenticated
gallery_footers	anon + auth	authenticated	authenticated	authenticated
performance_code_assignments	public	authenticated	authenticated	authenticated
result_edit_log	is_admin()	—	—	—

Storage RLS

Single bucket: `photos` . SELECT is public. INSERT/UPDATE/DELETE require `is_admin()` and folder must be one of: templates, spotlight, students, footers, frames.

Admin Panel — Features & Workflows

Dashboard (/admin)

Displays live statistics (total participants, teams, categories, programmes via row counts), a team points leaderboard sorted by total points across all published results, and individual points leaderboard by student category (Minor, HS, Premier, Sub Junior, Junior) with per-category tabs.

Programme Management (/admin/programmes)

Capabilities

- Add/Edit/Delete programmes with fields: name, category, programmeType (On-stage/Off-stage), participationType (Individual/Group), result number
- Toggle publish status (isPublished) — controls public result visibility
- Clear result — resets placements, marks as not finished
- Bulk import from CSV/Excel/PDF/OCR via ProgrammeBulkImportModal
- Filter by category, programme type, participation type
- Paginate (25/50/100/200/All)
- View enrolled participants modal per programme

Workflow: Creating a Programme

1. Click "Add Programme"
2. Enter name, select category, programme type, participation type
3. Optionally assign result number (or auto-assign)
4. Save — placeholder result row is auto-created via `ensureResultMasterRow()`
5. Programme appears in admin list and (if published) in public results

Workflow: Publishing Results

1. Judge submits result (locked + has data) via /judges/results
2. Admin reviews in /admin/results (read-only preview)
3. Admin toggles "Published" on the programme in /admin/programmes
4. Result becomes visible on /results and /results/:id
5. Student sees result on their dashboard

Team Management (/admin/teams)

View team scores (auto-calculated from published results via sync_team_totals trigger). Edit team name and color (16 preset colors). Expandable TeamBreakdown component shows per-category point breakdown and placement details.

Participant Management (/admin/students)

Capabilities

- Add/Edit/Delete participants with fields: name, chest number, category, team, photo, programme enrollment
- Photo upload to Supabase Storage (students/ folder)

- Bulk import via BulkImportModal (CSV/Excel/PDF/OCR)
- Filter by team and category
- View enrolled programmes modal

Category Management (/admin/categories)

CRUD for the categories table. Controls which categories appear in dropdowns/filters across the app. Default seed data: Minor(1), HS(2), Premier(3), Sub Junior(4), Junior(5), General Cat-A(6), General Cat-B(7).

Results Viewer (/admin/results)

Read-only results preview. Programme-centric list: every programme gets a row with result data. Shows placement rows (1st/2nd/3rd) with chest number, name, team, points, grade. Search by name/result number. Status badges: Published / Awaiting Publish / No result.

Gallery Management (/admin/spotlight)

Upload images (up to 10 per batch) to Supabase Storage. Add captions, assign to albums. Toggle "Hero banner" status (appears in home hero carousel). Edit caption/album. Delete images.

Gallery Footer Management (/admin/frames/footer)

Upload transparent PNG frames. Create/Edit/Delete footer overlays. Set one as "active" — enforced by trigger to ensure at most one active footer. Active footer is composited onto gallery photos at display time.

Student Panel — Features & Workflows

Login (/student/login)

Students authenticate with chest number + password. The system uses a custom session mechanism (not Supabase Auth) with the following properties:

- Credentials checked against `student_credentials` table, with fallback to common password `israfest2026`
- Session stored in both `localStorage` and the `students` table (dual storage)
- 8-hour session expiry
- Single-device enforcement: if `sessionActive=true` in DB, login is rejected with `'already_logged_in_elsewhere'`

Dashboard (/student/dashboard)

Profile Display

- Student name, chest number, category, team
- Profile photo (or colored initial avatar fallback)
- Circular progress ring showing completed/total programmes

Programme Lists

- **Completed:** Programmes where the student has a placement in published results
- **Pending:** Enrolled programmes without results yet

Poster Download

1. Student clicks "Download Poster" button
2. Picker modal shows completed programmes
3. Selects a programme → opens `TemplatePosterModal` (if templates exist) or `ResultPoster` (3-theme fallback)
4. Downloads poster as PNG

Edit Details (Admin-Gated)

Students can self-edit their profile, but must first authenticate an admin session:

1. Student clicks "Edit Details"
2. Prompted for admin email + password
3. System calls `supabase.auth.signInWithPassword()` with admin credentials
4. On success, edit form opens with fields: photo, name, category, team, programme enrollment
5. Changes saved via Supabase client (now authenticated as admin)
6. Admin session signed out after edit closes

Why? Students operate as the `anon` role in Supabase. RLS policies on the `students` table require `authenticated` role for writes. The admin re-authentication provides the necessary credentials.

Logout

Clears both localStorage session and DB session state (`sessionActive=false`), then redirects to `/student/login`.

07 — Judge Panel — Features & Workflows

Login (`/judges/login`)

Uses the isolated `judgeClient` instance. Validates `app_metadata.role === 'judge'` after sign-in. Clears any existing session before new login.

Result Entry Panel (`/judges/results`)

Layout

- Category filter dropdown
- "Not Submitted" section: programmes awaiting results
- "Submitted Results" section: locked results (read-only unless re-verified)

Workflow: First-Time Result Entry

1. Judge selects a programme from "Not Submitted" list
2. Clicks "Enter Result" to open the editor
3. Editor shows code letter dropdowns (populated from `performance_code_assignments` table)
4. Selects code letters and enters points (0-10) for each placement
5. Grade auto-calculated: 10=A+, 8-9=A, 6-7=B, 4-5=C
6. Can add multiple entries (dynamic row count)
7. Clicks "Save & Lock Result"
8. Result row updated with `locked: true, isFinished: true`
9. Programme moves to "Submitted Results" section

Workflow: Editing Locked Results (Re-Verification)

1. Judge clicks "Re-verify & Edit" on a locked result
2. Passes through re-verification flow (CAPTCHA + email + password)
3. Server-side verification via `judge_reverify_edit` RPC
4. Editor opens with existing entries pre-filled
5. Judge makes changes and saves
6. Audit log written to `result_edit_log` table

Public Pages — Features & Workflows

Home Page (/)

- **Hero Section:** 3D scene with floating GLB models (leaf, lens, seedpod), animated gradient background, parallax scroll effects
- **Team Standings:** Live leaderboard from published results with ember particle effects
- **Stats Blocks:** Animated counters for Teams, Days, Participants, Programmes
- **Gallery Preview:** First 8 spotlight images with footer overlay compositing
- **About Section:** Festival description for Rendezvous'26
- **Team Profiles:** 8 Corvion team member profiles with photos
- **Floating Nav:** Glassmorphism top bar with Login dropdown (Participant, Judges, Admin, Green Room) + ThemeToggle

Results Page (/results)

- Grid of published programmes with result number, grade, status
- Category filter dropdown
- "Unpublished" toggle to show/hide unpublished results
- Status badges: Result Available / Awaiting Result / Unfinished
- Click to view detail at /results/:id

Result Detail (/results/:id)

- Programme info (name, category, type)
- Medal placements (1st/2nd/3rd) with student avatar, name, chest number, team, points
- "View Posters" button opens TemplatePosterModal or ResultPoster fallback

Gallery (/gallery)

- Images grouped by album (sorted by newest first)
- Lightbox modal for full-size viewing
- Download button per image (with footer overlay composited into download)
- Back-to-home navigation

Bottom Navigation (Mobile)

Fixed bottom bar with 2 tabs: Home (/) and Results (/results). Supports rubber-band swipe gesture navigation. Hidden on admin/student/judge paths.

09 — Green Room & Lot Draw System

Access Gate (/lots)

PIN-protected access (hardcoded 6-digit code: `201219`). On success, sets `sessionStorage('lots_granted')`. No Supabase authentication involved.

Lot Draw (/lots/draw)

Three operational modes:

Mode 1: Topic Lotting

- Generates N numbered cards (1 to N)
- Shuffle and flip to reveal entry order numbers
- 3D CSS card-flip animation
- Max 60 cards

Mode 2: Code Letter Lotting

- Generates N lettered cards (A to Nth letter)
- Max 26 letters
- Same card-flip animation

Mode 3: Code Assignment (Green Room)

1. Select Category from dropdown
2. Select Programme from dropdown
3. View enrolled candidates with code letter dropdowns
4. Assign unique code letters (A, B, C...) to each candidate
5. Save → upserts to `performance_code_assignments` table with conflict resolution on (programme_id, code_letter)

This system enables **blind grading** — judges see only code letters, not student names.

Poster Generation System

Architecture

The poster system consists of four layers:

Layer	File(s)	Responsibility
Data Model	src/utils/posterTemplates.js	Template definitions, field mappings, data resolution, storage
Rendering	src/components/PosterStage.jsx	Interactive canvas with drag, resize, rotation, grid, snap guides
Editor UI	src/pages/admin/AdminPosterEditor.jsx	3-panel visual editor (Layer Library, Live Preview, Config)
Export	html2canvas + file-saver	DOM-to-canvas capture at 2x scale → PNG download

Template Model

- **Canvas:** 1080×1080 (standard 1:1) or legacy 720×960
- **Types:** `result` (programme winners) and `standings` (team leaderboard)
- **Elements:** Positioned text layers with optional data field bindings
- **Repeatable fields:** `{i}` placeholder renders multiple rows (one per placement/team)
- **Backgrounds:** Solid color, gradient (9 presets), or uploaded image

Data Field Mappings

Field Key	Template Type	Resolves To
<code>programme.name</code>	result	Programme name
<code>result.resultNo</code>	result	Result number (with "Result #" prefix)
<code>placement.{i}.name</code>	result	Winner name at position i
<code>placement.{i}.chestNo</code>	result	Chest number at position i
<code>placement.{i}.team</code>	result	Team name at position i
<code>placement.{i}.points</code>	result	Points at position i
<code>placement.{i}.grade</code>	result	Grade at position i
<code>team.{i}.name</code>	standings	Team name at rank i
<code>team.{i}.points</code>	standings	Team points at rank i

Poster Export Workflow

1. Admin designs template in AdminPosterEditor (or uses default)
2. Template saved to `poster_templates` table (with localStorage fallback)
3. User clicks "Export Poster" in editor OR "View Posters" on public result page
4. `buildPosterSource()` creates a value map from live programme/result data
5. `PosterStage` renders template with real data injected into field bindings

6. `html2canvas` captures the rendered DOM at 2x scale

7. `file-saver` triggers PNG download

Cumulative Team Poster (AdminResultPoster)

1. Admin enters programme count (multiples of 5)
2. System sums points per team across first N published programmes
3. Renders a green gradient poster with team rankings
4. Captures via `html2canvas`, stores in `localStorage`
5. Admin can toggle publish/unpublish for student download area

Legacy Poster System (ResultPoster)

Fallback when no templates exist. Renders 3 poster variants (Classic, Vibrant, Minimal) with programme name, result number, 1st/2nd/3rd winners, and points. Each captured via `html2canvas` at 2x scale.

11 — Print Sheet System

Admin Print (/admin/print)

Two tabs: Programmes (Sign + Valuation sheets) and Results (Result sheets).

Sheet Types

Sheet	Layout	Columns	Data Source
Sign Sheet	2 per A4 page	Chest No, Name, Code Letter, Signature	Enrolled participants + code assignments
Valuation Sheet	2 per A4 page	Code Letter, Grade, Price	Blank rows for hand-writing
Result Sheet	4 per A4 page	Chest No, Name, Team, Code, Grade, Price, Point	Locked judge results

Print Workflow

1. Admin selects sheet type and programmes
2. Multi-select with max 8 sheets constraint
3. Preview paginated A4 print layout
4. Trigger browser print dialog via `window.print()`

Data Import System

Supported Formats

Format	Library	Extension
CSV	PapaParse 5.6.0	.csv
Excel	SheetJS (xlsx) 0.18.5	.xlsx, .xls
PDF	pdfjs-dist 6.2.108	.pdf (text extraction)
Image (OCR)	Tesseract.js 7.0.0	.png, .jpg, etc.

Import Targets

Target	Modal	Fields
Participants	BulkImportModal	name, chestNo, category, team
Programmes	ProgrammeBulkImportModal	name, category, type, participationType, resultNo

Import Pipeline

1. User selects file via FileImportModal
2. `parseFileToRows()` universal parser detects format and extracts rows
3. Smart header detection using `HEADER_KEYWORDS` mapping
4. `validateParticipants()` or `validateProgrammes()` checks:
 - Required fields present
 - Duplicate chest numbers (participants)
 - Unknown categories/teams (validated against DB)
5. Preview displayed to user with validation errors highlighted
6. On confirmation, batch-inserted into Supabase

Utility Modules

Module	Lines	Purpose
importParsers.js	130	Universal file parser with tab/space table detection, PDF text extraction with Y/X coordinate layout, OCR with progress callback
bulkImportParser.js	~120	Participant-specific import with validation
programmeBulkParser.js	101	Programme-specific import with validation

13 — Gallery & Media System

Gallery Management

Upload (AdminSpotlight)

- Upload images (up to 10 per batch) to Supabase Storage `spotLight/` folder
- Each image has: `imageURL`, `caption`, `album`, `isFeatured` (hero banner toggle)
- Images grouped by album in public gallery (default album: "Spotlight")

Footer Overlays (AdminGalleryFooters)

- Upload transparent PNG frames to Supabase Storage `footers/` folder
- Create/Edit/Delete footer overlays
- Set one as "active" — enforced by `enforce_single_active_footer()` trigger
- Active footer composited onto gallery photos at display time (canvas-based, does NOT modify original)

Image Compositing Pipeline

1. Gallery photo displayed → `imageCompositor.js` loads base image and active footer
2. Canvas-based: footer placed at 25% height from bottom, 3.5% offset
3. Result cached as JPEG data URL in memory (`Map` cache keyed by `base|||footer`)
4. Downloads also go through this pipeline — footer baked into downloaded file

Storage Paths

Folder	Used By
<code>templates/{id}/bg-*</code>	Poster template background images
<code>spotlight/{ts}_{i}_{name}</code>	Gallery/spotlight images
<code>students/{ts}_{name}</code>	Student profile photos
<code>footers/{ts}_{name}</code>	Gallery footer overlay images
<code>frames/{ts}_{rand}.{ext}</code>	Frame images

Database Schema & Data Model

programmes

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
name	text		Programme name
category	text		Category name (e.g. "Minor")
isFinished	boolean	false	Event has ended
isPublished	boolean	false	Result released publicly
resultId	text		Legacy FK reference
programmeType	text		"On-stage" / "Off-stage"
participationType	text		"Individual" / "Group"
createdAt	timestampz	now()	Creation timestamp

teams

Column	Type	Default	Description
id	text (PK)		Team identifier
name	text		Display name
color	text		Team color
firstPlaceCount	bigint	0	Gold medals
secondPlaceCount	bigint	0	Silver medals
thirdPlaceCount	bigint	0	Bronze medals
totalPoints	bigint	0	Auto-synced by trigger

students

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
name	text		Student name
class	text		Category (e.g. "Minor")
team	text		Team ID reference
photoURL	text		Profile photo URL
programmels	jsonb		Array of programme IDs
chestNo	text		Display chest number
createdAt	timestampz	now()	Creation timestamp

sessionActive	boolean	false	Custom session tracking
sessionExpiresAt	timestamptz		Session expiry
sessionToken	text		Session token

results

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
name	text		Programme name (denormalized)
programmeld	text		FK to programmes.id
first	jsonb		{ studentId, name, points, grade, prize, code }
second	jsonb		Same shape as first
third	jsonb		Same shape as first
entries	jsonb	[]	Array of all placed entries (flexible)
updatedAt	timestamp		Last update time
resultNo	integer	0	Display number for ordering
locked	boolean	false	Judge-submitted = locked; immutable
isFinished	boolean	false	Synced from programme

categories

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
name	text (UNIQUE)		Category name
sortOrder	integer	0	Display ordering
createdAt	timestamptz	now()	Creation timestamp

spotlight

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
imageUrl	text		Image URL
caption	text		Image caption
order	bigint	0	Display ordering
isFeatured	boolean	false	Hero banner toggle
album	text		Gallery grouping
uploadedAt	timestamptz	now()	Upload timestamp

poster_templates

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier

name	text	'Untitled Template'	Template name
type	text	'result'	"result" or "standings"
canvas	jsonb		Canvas dimensions/config
background	jsonb		Background config
elements	jsonb	[]	Text/element layers
teamsToShow	integer	8	Standings teams to display
createdAt	timestamptz	now()	Auto-touched by trigger
updatedAt	timestamptz	now()	Auto-touched by trigger

gallery_footers

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
name	text	'Footer'	Footer name
image_url	text		Image URL
is_active	boolean	false	Single-active enforced by trigger
created_at	timestamptz	now()	Creation timestamp
updated_at	timestamptz	now()	Auto-touched by trigger

performance_code_assignments

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
programme_id	text (NOT NULL)		FK to programmes
participant_id	uuid (NOT NULL)		FK to students(id) ON DELETE CASCADE
code_letter	text (NOT NULL)		Blind-grading letter code
created_at	timestamptz	now()	Creation timestamp
UNIQUE: (programme_id, code_letter)			

student_credentials

Column	Type	Description
student_id	uuid	FK to students(id)
password	text	Password (plaintext)

judge_captcha_challenges

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
judge_id	uuid (NOT NULL)		auth.users.id of judge
captcha	text (NOT NULL)		6-char code
expires_at	timestamptz	now() + 5 min	Expiry timestamp

used	boolean	false	Single-use flag
------	---------	-------	-----------------

result_edit_log

Column	Type	Default	Description
id	uuid (PK)	gen_random_uuid()	Unique identifier
result_id	uuid		Result being edited
programme_id	text		Programme reference
programme_name	text		Programme name
judge_email	text		Judge who edited
edited_at	timestamptz	now()	Edit timestamp
old_first/old_second/old_third	jsonb		Previous state
new_first/new_second/new_third	jsonb		New state

Supabase RPCs & Functions

Trigger Functions

Function	Type	Trigger	Purpose
<code>sync_team_totals()</code>	SECURITY DEFINER	AFTER INSERT/UPDATE/DELETE ON results (FOR EACH STATEMENT)	Recalculates teams.totalPoints by summing points from all latest results per programme
<code>touch_poster_template_updated_at()</code>	Trigger function	BEFORE INSERT/UPDATE ON poster_templates	Auto-sets updatedAt = now()
<code>enforce_single_active_footer()</code>	Trigger function	BEFORE INSERT/UPDATE ON gallery_footers	Ensures at most one row has is_active = true

RPC Functions

`judge_credentials(p_judge_email, p_judge_password)`

Property	Value
Type	SECURITY DEFINER, returns jsonb
Grants	authenticated, anon
Purpose	Server-side password verification against auth.users.encrypted_password
Returns	{ valid: true, judge_id, email, role } or { valid: false, error: "invalid_credentials" }

`judge_create_captcha()`

Property	Value
Type	SECURITY DEFINER, returns jsonb
Auth gate	Checks app_metadata.role = 'judge' inside function
Purpose	Generates 6-char captcha, stores in judge_captcha_challenges with 5-min expiry
Returns	{ challenge_id, captcha, expires_at } or { error: "not_authorized" }

`judge_reverify_edit(...)`

Property	Value
Type	SECURITY DEFINER, returns jsonb
Parameters	p_challenge_id, p_captcha, p_judge_email, p_judge_password, p_programme_id, p_programme_name, p_result_no, p_first, p_second, p_third, p_entries
Purpose	The ONLY secure write path for locked results. Verifies: (1) JWT role is judge, (2) email matches, (3) password valid via crypt(), (4) captcha valid/unexpired/unused, (5) at least one placement. Then upserts result, sets locked=true, writes audit log.

`admin_set_result_no(p_programme_id, p_programme_name, p_result_no)`

Property	Value
Type	SECURITY DEFINER, returns jsonb
Purpose	Lets admin edit a programme's displayed result number. Creates placeholder result row if none exists.

`admin_delete_results_for_programme(p_programme_id)`

Property	Value
Type	SECURITY DEFINER, returns jsonb
Purpose	Bypasses RLS to delete ALL result rows for a given programme. Used when un-finishing, deleting, or clearing results.

RPC Call Map

RPC	Called From	Client
judge_create_captcha	JudgesResults.jsx	judgeClient
judge_reverify_edit	JudgesResults.jsx	judgeClient
admin_delete_results_for_programme	AdminProgrammes.jsx	supabase
admin_set_result_no	AdminProgrammes.jsx	supabase

16 — Design System

Theme System

Class-based dark mode (`darkMode: 'class'` in Tailwind config). Toggled via `data-theme` attribute on `<html>` . Theme persisted in localStorage under key `artfest-theme` . A blocking script in `index.html` reads this on page load to prevent flash-of-wrong-theme.

Color Tokens (CSS Variables)

Token	Light Value	Dark Value
--bg-page	#F5FAF3	#061A0D
--text-main	#115F32	#D4FFB8
--text-muted	#52705D	#8ED06C
--card-white	FFFFFF	#0B2A17
--card-lavender	#D4FFB8	#0B2A17
--accent-purple	#115F32	#115F32
--accent-purple-deep	#228C22	#228C22
--border-subtle	rgba(17,95,50,0.25)	rgba(17,95,50,0.4)
--admin-sidebar-bg	#011E11	#0d3520
--admin-main-bg	FFFFFF	#011E11

Typography

Primary Font: Sora (Google Fonts, weights 300-800) — set as global body font. All Tailwind font utility classes (font-poppins, font-inter, font-display, font-playfair, font-brand) map to Sora.

Custom Font: CSGENERAL-REGULAR_DEMO — used exclusively for the "Corvion" brand name.

Poster Fonts: Sora, Montserrat, Inter, Roboto, Serif (Georgia), Monospace (Courier New), CSGENERAL.

CSS Component Classes

Class	Purpose
.floating-nav	Glassmorphism floating navigation bar
.nav-glass-vessel	Liquid-glass bottom nav container
.glass-chip	Glass-effect button/chip
.bento-white-card / .bento-dark-card	Card variants with hover lift
.lot-card / .lot-card-inner / .lot-card-face	3D flip card (lottery draw)
.cta-gradient	Primary CTA gradient button
.admin-sidebar / .admin-main-content	Admin layout containers
.lb-1, .lb-2, .lb-3	Leaderboard row styles (gold/silver/bronze)
.reveal / .reveal-visible	Scroll-reveal animation
.starfield / .star	Animated star background

Project Structure & Deployment

Source Directory Structure

```

src/
├─ App.jsx                Root component (BrowserRouter + Routes)
├─ main.jsx               Entry point (ToastProvider wrapper)
├─ index.css              Design system, CSS variables, animations
├─ assets/                Fonts, images, hero assets
│   └─ CSGENERAL-REGULAR_DEMO.OTF
├─ components/            26 reusable UI components
│   ├─ BottomNav.jsx      Mobile bottom navigation
│   ├─ BulkImportModal.jsx CSV/Excel/PDF bulk import
│   ├─ CursorGlow.jsx     Mouse-following ambient glow
│   ├─ FileImportModal.jsx Generic file import dialog
│   ├─ FilterDropdown.jsx Filter dropdown UI
│   ├─ FloatingCards.jsx  3D floating card carousel
│   ├─ GalleryFooterOverlay.jsx Footer overlay on photos
│   ├─ HeroAnimation.jsx  Hero section animation
│   ├─ HeroScene.jsx      Three.js 3D hero scene
│   ├─ IsraLogo.jsx       ISRA brand logo
│   ├─ JudgesRoute.jsx    Judges auth guard
│   ├─ KebabMenu.jsx      Overflow menu
│   ├─ LoginControl.jsx   Login/logout dropdown
│   ├─ PhytoLoreHero.jsx  Alternative hero component
│   ├─ PosterGeneratorModal.jsx Poster generation dialog
│   ├─ PosterStage.jsx    Poster rendering canvas
│   ├─ ProgrammeBulkImportModal.jsx
│   ├─ ProtectedRoute.jsx Admin auth guard
│   ├─ ResultPoster.jsx   Legacy poster display
│   ├─ Starfield.jsx      Animated starfield
│   ├─ StudentAvatar.jsx  Student avatar display
│   ├─ TeamBreakdown.jsx  Team breakdown view
│   ├─ TemplatePosterModal.jsx Template selection modal
│   ├─ ThemeToggle.jsx    Light/dark theme toggle
│   └─ Toast.jsx          Toast notification system
├─ hooks/                 2 custom hooks
│   ├─ useCountUp.js      Animated number counter
│   └─ useScrollReveal.js IntersectionObserver scroll reveal
├─ pages/                 Page-level components
│   ├─ Home.jsx           Landing page
│   ├─ Gallery.jsx        Photo gallery
│   ├─ Results.jsx        Public results listing
│   ├─ ResultDetail.jsx   Individual result view
│   ├─ LotsAccess.jsx     Lottery access gate
│   ├─ LotsDraw.jsx       Lottery draw page
│   ├─ StudentLogin.jsx   Student login
│   ├─ StudentDashboard.jsx Student dashboard
│   └─ admin/             15 admin pages
│       ├─ AdminLogin.jsx
│       ├─ AdminLayout.jsx
│       ├─ AdminDashboard.jsx
│       ├─ AdminProgrammes.jsx
│       ├─ AdminTeams.jsx
│       ├─ AdminStudents.jsx
│       ├─ AdminSpotlight.jsx
│       ├─ AdminGalleryFooters.jsx
│       ├─ AdminCategories.jsx
│       ├─ AdminPrint.jsx
│       └─ AdminResults.jsx

```

```
| | | | | AdminResultPoster.jsx
| | | | | AdminPosterTemplates.jsx
| | | | | AdminPosterEditor.jsx
| | | | | AdminLots.jsx
| | | | |
| | | | | judges/                2 judge pages
| | | | | | JudgesLogin.jsx
| | | | | | JudgesResults.jsx
| | | | |
| | | | | supabase/            Backend integration
| | | | | | client.js          3 Supabase client instances
| | | | | | queries.js         808 lines of DB query functions
```

Deployment Configuration

Setting	Value
Host	Vercel
Build Command	vite build
Output Directory	dist
SPA Rewrite	/((?!assets/).*) → /index.html
Live URL	isra-rendezvous.vercel.app
Supabase Project	lvkehbuscijtovbrcikq.supabase.co

Entry Point Chain

```
index.html
| | | | | localStorage theme flash-prevention script
| | | | | Google Fonts (Sora 300-800)
| | | | | #root
| | | | | | src/main.jsx
| | | | | | | <StrictMode>
| | | | | | | | <ToastProvider>
| | | | | | | | | <App />
| | | | | | | | | | <BrowserRouter>
| | | | | | | | | | | <Starfield />
| | | | | | | | | | | <CursorGlow />
| | | | | | | | | | | <Routes> (all page routes)
| | | | | | | | | | | <BottomNav />
```

localStorage Keys

Key	Purpose
artfest-theme	Theme preference (light/dark)
student_id	Current student ID
student_session_<id>	Student session token
student_session_expires_<id>	Student session expiry timestamp

sessionStorage Keys

Key	Purpose
lots_granted	Green room access flag (set on PIN entry)

Global Components (Rendered on Every Page)

Component	Purpose
-----------	---------

Starfield	Animated twinkling star background
CursorGlow	Mouse-following ambient glow (hidden on touch devices)
BottomNav	Mobile bottom navigation (hidden on admin/student/judge paths)
ToastProvider	Global toast notification context